

MLOps & Deployment

A Practical Mini-Guide with Examples

1. Docker Basics

Docker packages an application together with all its dependencies into a portable **image**. Running that image produces a **container** — an isolated, consistent environment that behaves the same on any machine (your laptop, a server, or the cloud). This solves the classic "it works on my machine" problem.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

# Build and run:
docker build -t my-ml-api .
docker run -p 8000:8000 my-ml-api
```

2. FastAPI

FastAPI is a modern Python web framework for building APIs quickly. It's fast, type-safe (using Pydantic), and automatically generates interactive documentation (Swagger UI at **/docs**).

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def health_check():
    return {"status": "ok"}

# Run with: uvicorn main:app --reload
```

3. REST API Creation (for ML models)

A REST API exposes your trained model over HTTP so other applications can send data and receive predictions. Typically a **POST** endpoint accepts input features and returns the model's output as JSON.

```
from pydantic import BaseModel

class InputData(BaseModel):
    feature1: float
    feature2: float

@app.post("/predict")
def predict(data: InputData):
    result = model.predict([[data.feature1, data.feature2]])
    return {"prediction": result.tolist()}
```

4. ML Model Serialization (pickle / joblib)

Serialization saves a trained model's state to disk (weights, parameters) so it can be reloaded later without retraining. **joblib** is preferred for scikit-learn models since it handles large NumPy arrays efficiently; **pickle** is Python's general-purpose serializer.

```
import joblib
from sklearn.linear_model import LogisticRegression

model = LogisticRegression().fit(X_train, y_train)
joblib.dump(model, "model.joblib")  # save

# later, inside the API service:
model = joblib.load("model.joblib")  # load
```

5. GitHub Actions Overview

GitHub Actions automates workflows — such as testing, building, and deploying code — triggered by events like a **push** or **pull_request**. Workflows are defined in YAML files under **.github/workflows/**.

```
# .github/workflows/ci.yml
name: CI
on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install -r requirements.txt
      - run: pytest
```

6. CI/CD Concepts

Continuous Integration (CI) automatically builds and tests code every time it changes, catching bugs early. **Continuous Delivery/Deployment (CD)** automatically ships tested code to staging or production, reducing manual release effort.

Typical ML pipeline:

```
push code
-> run unit tests
-> build Docker image
-> push image to registry
-> deploy to cloud environment
```

7. MLflow Overview

MLflow tracks machine learning experiments: parameters, metrics, artifacts, and models. This makes it easy to compare runs, reproduce results, and manage model versions.

```
import mlflow
with mlflow.start_run():
```

```
mlflow.log_param("C", 1.0)
model = LogisticRegression(C=1.0).fit(X_train, y_train)
acc = model.score(X_test, y_test)
mlflow.log_metric("accuracy", acc)
mlflow.sklearn.log_model(model, "model")
```

```
# View results: mlflow ui
```

8. Cloud Deployment Overview (AWS / Azure)

The common pattern: containerize the app with Docker, push the image to a cloud container registry, then run it on a managed compute service. AWS offers ECR + ECS/App Runner; Azure offers ACR + App Service/Container Apps.

```
# AWS example
aws ecr create-repository --repository-name my-ml-api
docker tag my-ml-api:latest <account>.dkr.ecr.<region>.amazonaws.com/my-ml-api
docker push <account>.dkr.ecr.<region>.amazonaws.com/my-ml-api
# then deploy the image via ECS / App Runner
```

```
# Azure equivalent
az acr build --registry myregistry --image my-ml-api .
az containerapp up --name my-ml-api --image myregistry.azurecr.io/my-ml-api
```

9. Final Project Presentation

Bringing everything together into one end-to-end MLOps project. A strong presentation structure covers the full lifecycle from data to deployment:

Step	What to show
1. Problem & Data	Define the problem statement and describe the dataset used.
2. Model Training	Train the model and track experiments/metrics with MLflow.
3. API Wrapping	Serialize the model (joblib) and serve it via a FastAPI REST endpoint.
4. Containerization	Package the API into a Docker image.
5. CI/CD Pipeline	Automate testing and builds with GitHub Actions.
6. Cloud Deployment	Deploy the container to AWS or Azure.
7. Live Demo & Q&A	Show a live prediction request and answer questions.